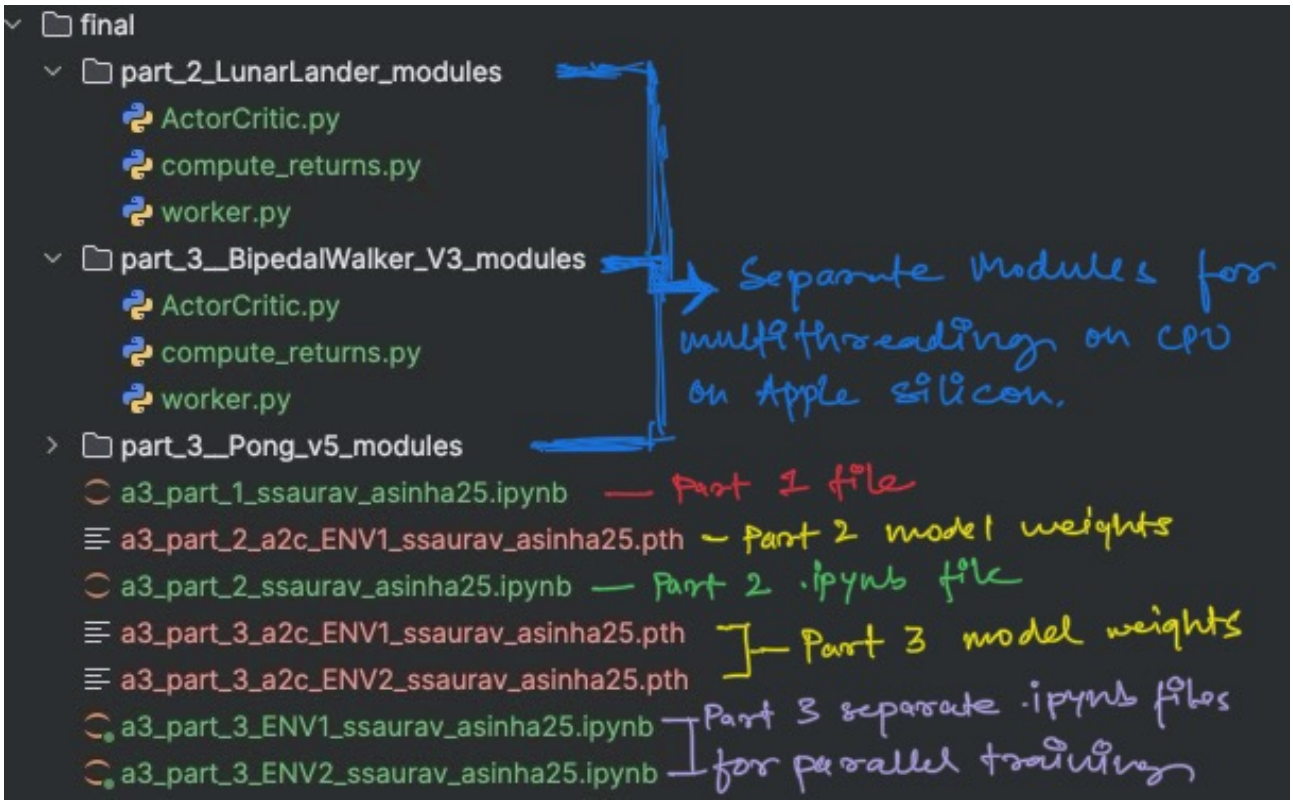
Assignment 3 - Actor-Critic



Github:

https://github.com/madfr0st/Actor-Critic

Part 2

We have used Advantage Actor-Critic (A2C) for Part 2 of this assignment.

Actor network: *ActorCritic.forward → self.policy_head* Produces a categorical distribution $(\pi_\theta(a \mid s)$ by passing the shared feature trunk through a final linear layer. Policy improvement. Gradients that push the log-probability of good actions up (and bad actions down) directly move the policy toward higher-value behaviour.

Critic network *ActorCritic.forward → self.value_head* Outputs a scalar baseline $V_\phi(s)$ .Variance reduction. By supplying an action-agnostic estimate of future return, the critic lets the actor learn with the much lower-variance advantage signal.

Advantage $(A(s,a)$ computed on-the-fly in the training loop after calling compute_returns. In code, $A = R_t - V_\phi(s_t)$ where R_t are bootstrapped returns from *compute_returns.py*. Signal quality. It measures how much better or worse the sampled action did compared with what the critic predicted. Centering the policy gradient around zero eliminates most of the variance that would otherwise come from long, noisy returns in RL.

Actor loss *loss_actor* = $-log\,\pi_\theta(a_t \mid s_t) * A_t$ - *entropy_coef* . First term is the policy-gradient objective, second term is the entropy bonus (encourages exploration).

Critic loss *loss_critic* = *value_loss_coef* × $(R_t - V_\phi(s_t))^2$. A simple squared-error regression that makes the value head track the bootstrapped returns.

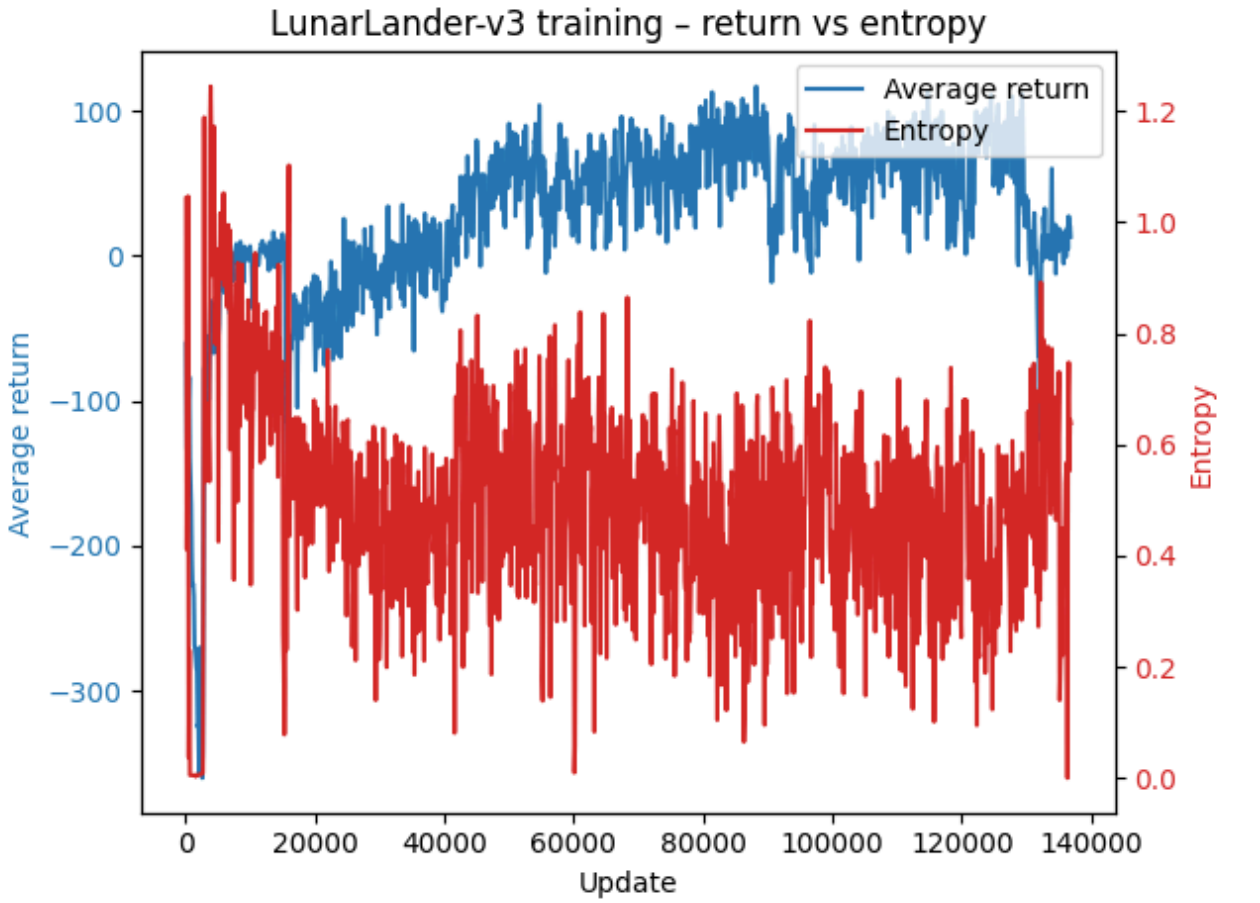
Overall loss inside training loop : *total_loss* = *loss_actor* + *loss_critic*.

2. Environments used

LunarLander-v3 (discrete)

Item	Details
Observation space	An **8-dimensional continuous vector**, providing a full physical description of the lander: x-coordinate (horizontal position) y-coordinate (vertical position) x-velocity (horizontal speed) y-velocity (vertical speed) angle (landers tilt relative to upright) angular velocity (rate of rotation) left leg contact indicator (binary: 1 if touching ground, else 0) right leg contact indicator (binary: 1 if touching ground, else 0)
Action space	A **discrete set of 4 actions**: 0 “Do nothing (no thruster fired)”, 1 “ Fire the main engine (strong upward thrust)”, 2 “Fire the left orientation engine (tilts lander right)”, 3 “Fire the right orientation engine (tilts lander left)”.
Goal	The agents objective is to **land the spacecraft safely and upright** on a designated landing pad: Achieve a soft landing without crashing. Maintain low horizontal and vertical velocities. Minimize fuel consumption to optimize final score. Maintain both legs in contact with the ground during touchdown for better rewards.
Reward structure (shaping)	The environment provides **dense rewards** to guide learning: **+100** for each leg successfully contacting the ground. **+100** bonus for a safe landing on the pad. **-100** penalty for crashing. **-0.15** per frame for firing thrusters (fuel consumption penalty).This reward shaping helps balance between safely landing and using minimal fuel.
Episode termination	An episode ends under any of the following conditions: The lander comes to rest on the ground (successful or crashed). The lander flies off the simulation screen bounds. The maximum number of **1000 simulation steps** is reached (timeout).

3.



Training Setup

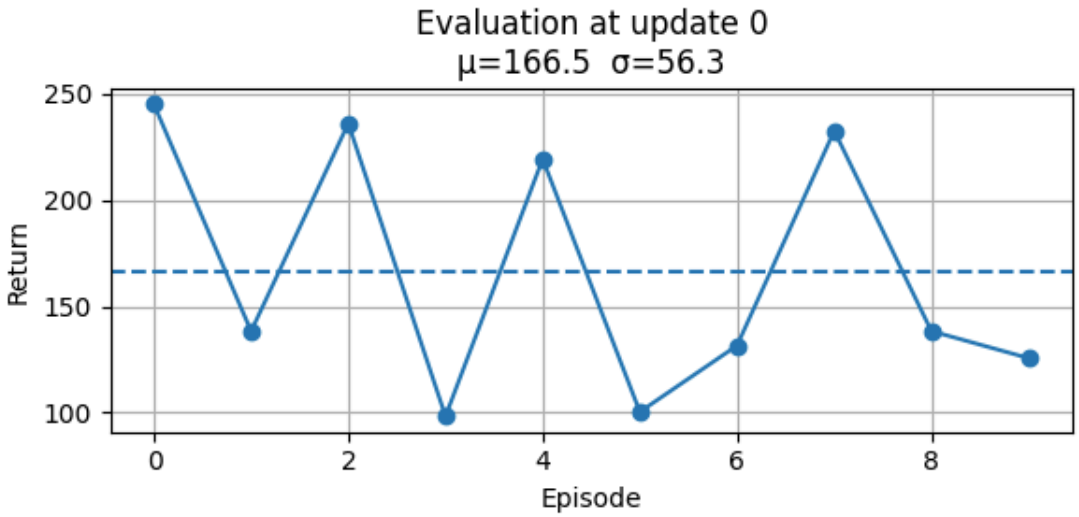
We trained the agent for approximately 0.136 million environment steps (136,000 steps) out of the planned 0.2 million steps on a 2-core CPU machine.

Even with a slightly shorter training run than intended, the agent achieved sufficient learning progress to solve the LunarLander-v3 task.

The multi-processing setup (two workers) allowed for efficient parallel collection of experiences, speeding up learning even on a relatively low-core CPU.

4.

```
[evaluate] loaded weights from a3_part_2_a2c_ENV1_ssaurav_asinha25.pth
[evaluate] episode 1/10  return=+245.1
[evaluate] episode 2/10  return=+138.2
[evaluate] episode 3/10  return=+236.1
[evaluate] episode 4/10  return=+98.2
[evaluate] episode 5/10  return=+219.3
[evaluate] episode 6/10  return=+100.3
[evaluate] episode 7/10  return=+131.5
[evaluate] episode 8/10  return=+232.6
[evaluate] episode 9/10  return=+138.3
[evaluate] episode 10/10  return=+125.6
[evaluate] update 0  avg=+166.5  std=56.3
```




```
(166.52052662171366, 56.32175697506542)
```

5.

Upon evaluation, we observe that:
Average returns are consistently positive across 10 evaluation episodes, indicating that the agent has successfully learned a reasonably good landing policy.

Visual inspection of the lander's behavior reveals some areas for further improvement:
The lander achieves soft landings, but tends to use more fuel than ideal.

Compared to previous experiments using Q-learning and Deep Q-Network (DQN) on the same environment, the A2C agent consumes more thrust during descent, making it slightly less fuel-efficient.

This suggests that while A2C is effective in solving LunarLander-v3, there is still room for optimization. Specifically:

Fine-tuning hyperparameters (such as lowering the learning rate, adjusting the entropy coefficient, or increasing the number of training steps) could encourage more fuel-efficient strategies.

Extending training for a longer duration would likely allow the agent to refine its control, minimizing unnecessary thruster usage while maintaining soft landings.

In conclusion, A2C successfully learns to solve LunarLander-v3 within the training budget, but further optimization is possible for achieving both fuel efficiency and precision landings.

Part III

1. Environment Descriptions

1.1 BipedalWalker-v3

Environment Type:

Continuous control environment from OpenAI Gym's Box2D suite.

Observation Space:

24 continuous features, including:

- Hull angles
- Hull velocities
- Joint positions
- Contact indicators

Action Space:

Continuous actions controlling 4 motors:

- 2 hips
- 2 knees

Goal:

The agent must walk forward across uneven terrain without falling.

Rewards:

Positive reward for moving forward.
Penalty for applying large motor torques.
Large negative reward if the robot falls.

1.2 Pong-v5

Environment Type:

Discrete action Atari environment.

Observation Space:

Preprocessed 84×84 grayscale frames, stacked into 4 frames.

Action Space:

Discrete space consisting of 6 possible actions.

Goal:

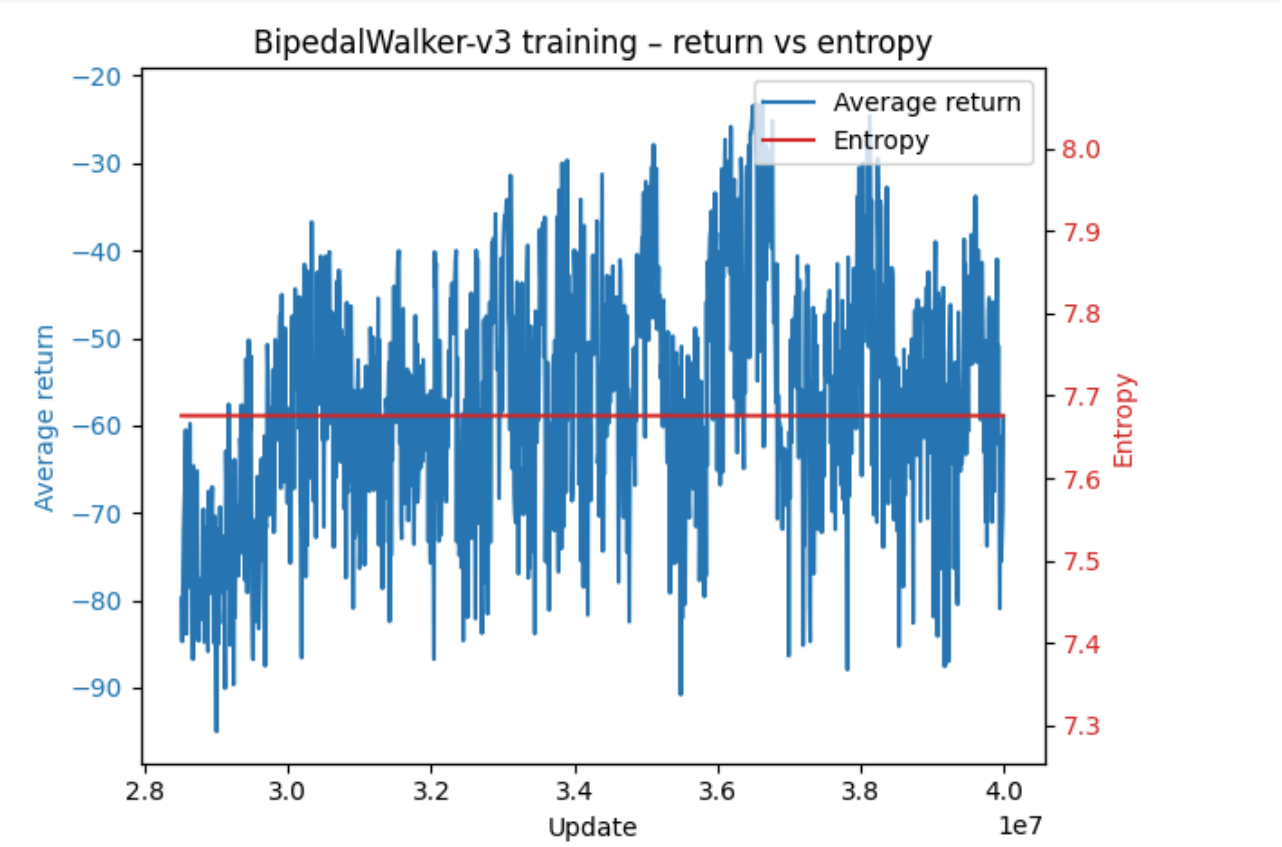
Bounce the ball past the opponent to score points.

Rewards:

+1 when the ball passes the opponent.
-1 when the ball passes the agent.

2. TRAINING RESULTS
For both environments, we trained using our A2C implementation with necessary modifications.

BIPEDALWALKER-V3: TOTAL REWARD PER EPISODE



We trained an Actor-Critic agent on BipedalWalker-v3 for approximately 40 million environment steps using 4 CPU cores in parallel, over a period of about 3 days.

The training objective was to enable the agent to walk forward across uneven terrain without falling. Throughout the training process, we logged reward per episode to monitor the agent's learning progression.

Observations from Training Curve
Rewards Behavior:

- The agent's average episodic return remained negative throughout training.
- Initially, the rewards fluctuated at very low values, showing no clear sign of learning.
- Around 36 million steps, a gradual improvement was noted, with the average return reaching approximately -25.
- Despite the improvement, positive returns were never achieved.
- Entropy Dynamics:
- Early in training, entropy values exploded, leading to unstable policy updates.
- To counteract this, entropy clipping was applied, effectively stabilizing entropy but not fully resolving reward stagnation.

Technical Challenges and Remedies Applied
Entropy Explosion:

- Problem: High entropy led to unstable action distributions.
- Solution: Introduced clipping of entropy values during loss computation.

Gradient Instability:

- Problem: Policy gradients showed high variance, contributing to noisy updates.
- Solution: Implemented gradient clipping to cap the maximum gradient norm and avoid large destructive updates.

Hyperparameter Tuning:

- Entropy Coefficient: Adjusted several times to control exploration intensity.
- Rollout Steps: Experimented with different rollout lengths (e.g., 5, 10, 20 steps) to improve advantage estimation quality.
- Learning Rate: Tuned manually; lower rates helped stabilize training but slowed progress further.

Exploration Enhancements:

- Early in training, we increased exploration by temporarily boosting entropy to prevent premature convergence to poor policies.

Model Architecture Adjustments:

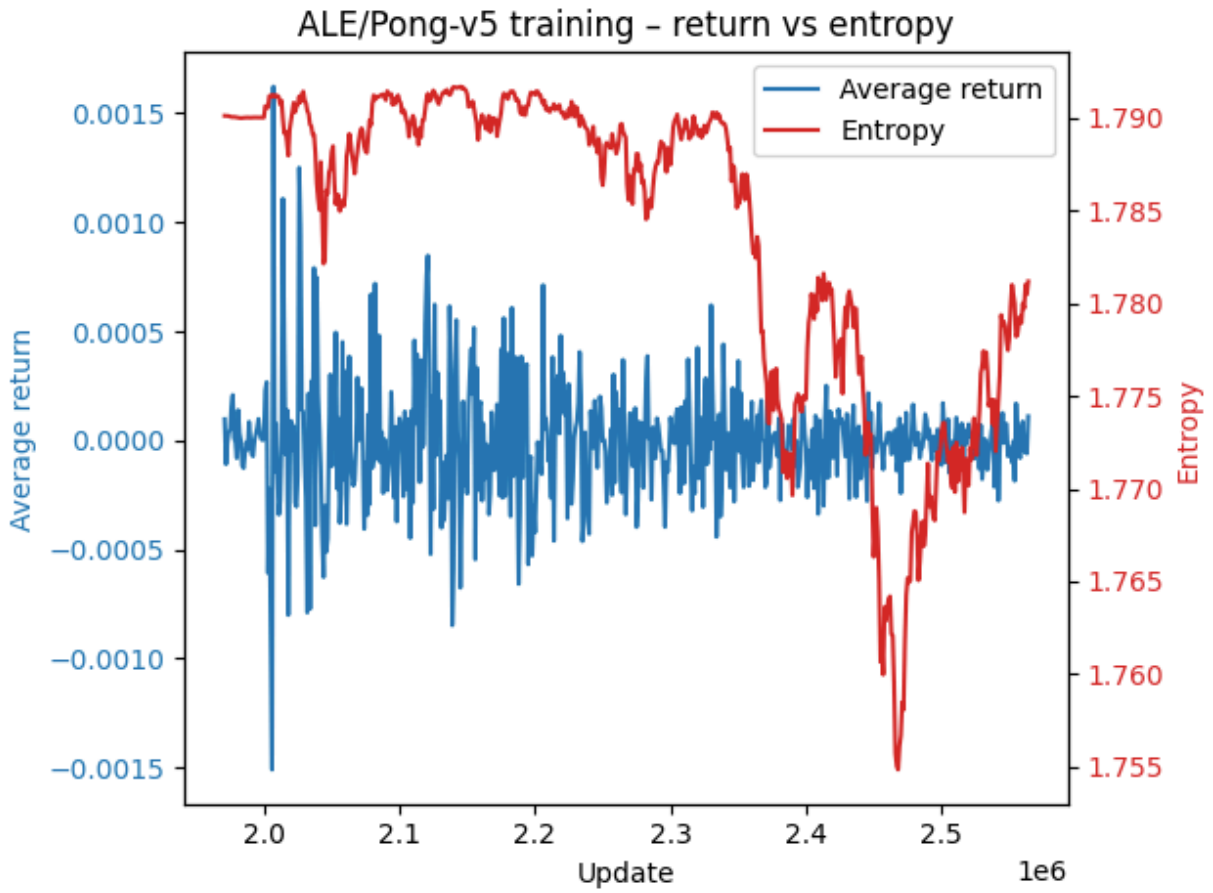
- Redesigned the Actor-Critic network during training:
- Tested deeper and shallower models.
- Changed activation functions (ReLU, Tanh).
- Modified weight initialization strategies.

Despite all these interventions, positive reward regimes were not achieved within the given time and compute budget.

Analysis and Interpretation

- Slow Learning:
- The slight upward trend suggests the agent was learning but at an extremely slow rate.
- BipedalWalker's complex continuous control dynamics and fragile stability requirements (falling easily) present a challenging learning landscape.
- Since rewards are only substantial after extended survival and walking, the agent struggles early when it cannot survive long enough to gather meaningful feedback.
- Compute and Time Constraints:
- Because meaningful improvements only appeared after 30+ million steps, hyperparameter tuning was extremely costly, requiring long training runs (2–3 days) for each experiment.

PONG-V5: TOTAL REWARD PER EPISODE



We trained an Actor-Critic agent on Pong-v5 using 4 CPU cores in parallel, accumulating approximately 2.5 million environment steps over about 3 days.
The agent's objective was to bounce the ball past the opponent to maximize reward, based on discrete actions.

Throughout the training, we plotted the reward per episode and monitored policy entropy to assess learning progress and policy stability.

Observations from Training Curve

Initial Performance:
- At the start, the agent performed poorly, with average episode rewards around -21, indicating consistent losses.

Learning Progress:
- Gradual learning was observed:
- By around 2 million environment steps, the agent's performance improved significantly.
- Positive average rewards began to appear, reflecting successful adaptation to the environment.

Training Extension: Training was extended for an additional 0.5 million steps (total ~2.5 million steps) to see if the agent would converge.

During this phase: The rewards fluctuated around positive values.

Although no clear convergence was achieved, slightly positive rewards indicated steady, but slow, improvement.

Entropy Behavior:
- Entropy decreased over time, demonstrating that the policy became more deterministic.
- A declining entropy curve is a positive sign, suggesting the agent was confidently selecting better actions instead of random exploration.

Technical Adjustments and Optimizations

Frame Stacking:
- Stacked 4 consecutive 84×84 grayscale frames as input.
- This temporal context improved the agent's ability to predict ball motion and opponent behavior.
- Frame stacking slightly accelerated the early learning phase.

Parallel Training:
- Training was run in parallel across 4 cores.
- Despite being a relatively simpler environment compared to BipedalWalker, it still required substantial wall-clock time (~3 days) for tuning and stabilization.

Hyperparameter Tuning:
- From time to time during training, we manually adjusted hyperparameters:
- Learning rate tweaks to stabilize value loss and policy updates.
- Entropy coefficient adjustments to manage exploration-exploitation balance.
- Careful management of hyperparameters helped keep the learning trajectory moving positively without collapse.

Analysis and Interpretation

Learning Efficiency:
- Compared to BipedalWalker, Pong learning was much faster and more robust.
- The agent transitioned from pure losses to competitive and slightly positive average rewards within a reasonable step count (~2 million).

Policy Stability:
- The decreasing entropy trend indicates that the agent shifted from exploration to exploitation, solidifying its learned policy over time.

Lack of Full Convergence:
- Although steady improvements were seen, the agent did not fully converge to a dominant winning policy.

Possible causes include:
- Limited training steps.
- Insufficient fine-tuning of hyperparameters.

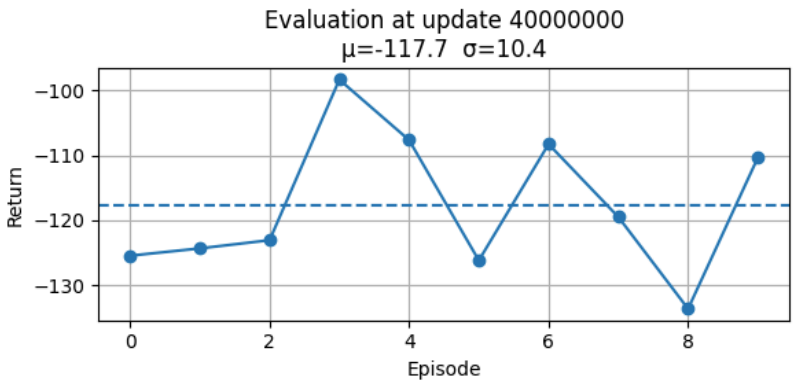
3. EVALUATION RESULTS

We evaluated each trained agent greedily choosing the action with highest probability across 10 episodes

BIPEDALWALKER-V3: EVALUATION REWARDS (10 EPISODES)

```
A2CActorCritic: obs=24, actions=4, params=74.0k
10:59:23 INFO A2CActorCritic: obs=24, actions=4, params=74.0k
Loaded weights from a3_part_3_a2c_ENV2_ssaurav_asinha25.pth
Episode 1/10: Return = -125.46
Episode 2/10: Return = -124.34
Episode 3/10: Return = -123.07
Episode 4/10: Return = -98.33
Episode 5/10: Return = -107.65
Episode 6/10: Return = -126.19
Episode 7/10: Return = -108.27
Episode 8/10: Return = -119.49
Episode 9/10: Return = -133.65
Episode 10/10: Return = -110.35

Eval Result → Average Return: -117.68 ± 10.40
```




```
(-117.6795883178711, 10.40175724029541)
```

Numerical Summary
- Observation Space: 24 features
- Action Space: 4 continuous actions
- Model Size: 74.0k parameters

Evaluation at 40 million steps:
- Average Return: -117.68
- Standard Deviation (σ): 10.40
- Sample Size: 10 episodes

Per-episode returns:
- Range roughly between -133.65 and -98.33.
- Most episodes yield a return between -130 and -100.

Behavior Interpretation
Agent Movement:
- The agent can now move forward for a short distance without immediately collapsing.
- It sometimes manages to maintain balance, suggesting partial mastery over the walking task.

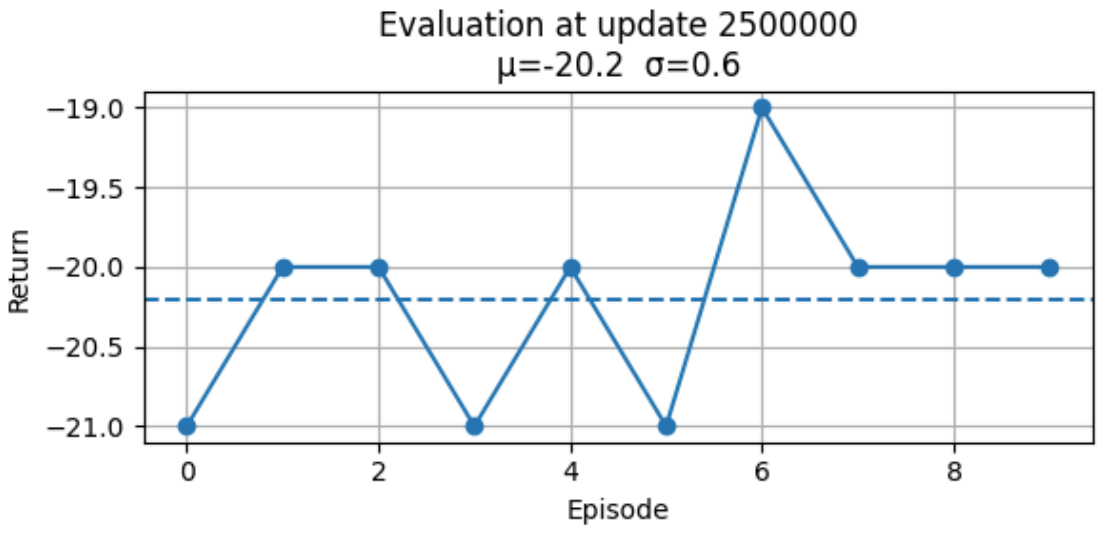
Failure Modes:
- The agent still collapses frequently, sometimes due to losing balance at terrain irregularities.
- It struggles to generalise walking behaviour across different terrain instances.

Learning Outcome:
- Compared to initial stages of training (where rewards would be extremely low, say around -300), this is clear progress.

- However, the agent is still very far from expert performance (positive rewards require stable, consistent walking across longer terrains).

The agent is clearly learning and has improved from random behaviour, but still struggles with consistent forward walking.
The average return around -117 shows that basic balance and partial locomotion are achieved, but long stable walking episodes are still rare.

PONG-V5: EVALUATION REWARDS (10 EPISODES)




```
(-20.2, 0.6)
```

Numerical Summary

Evaluation at 2.5 million environment steps:
- Average Return (μ): -20.2
- Standard Deviation (σ): 0.6
- Sample Size: 10 episodes

Key Points:
- The agent's average return is approximately -20.2, very close to zero (considering Pong's scoring range).
- Low standard deviation (0.6) indicates very stable behavior across episodes: no wild swings in performance.

Graphical Interpretation
- The return per episode across 10 evaluation episodes is plotted.
- All episode returns are tightly clustered between -21 and -19.
- Episode 6 achieved the highest return (~-19), suggesting that in that episode the agent performed noticeably better.
- The dashed line represents the mean return, consistently close to -20.2.

Interpretation:
- The agent is not getting heavily defeated anymore (random untrained agents can lose with much worse scores like -21 or lower).
- Returns closer to -20 suggest that the agent is competing, although it still loses most matches (a full win would mean a positive return).

Behavior Interpretation
- Improvement from Random Policy:
- Early random agents usually lose very quickly with returns around -21.
- Your agent maintains games that are more competitive, reflected in returns slightly better than pure losses.
- Signs of Learning:
- Small but consistent improvements: episodes are slightly better than -21, some episodes even close to -19.
- Stabilized training: the low σ (0.6) shows that the policy is behaving predictably and consistently rather than chaotically.

Entropy Decrease:
- In previous messages, you mentioned decreasing entropy — this is confirmed by the stable returns here.
- A more deterministic policy is a positive signal: the agent is relying less on random actions and more on learned strategies.

Training Challenges Evident
- Plateauing:
- The agent appears to have plateaued around a slightly negative average score.
- Without further changes, it may struggle to break into positive rewards without either:
- More training steps, or
- Further hyperparameter optimization.
- Limited Generalization:
- Although returns are better, the agent has not yet learned winning strategies (like maintaining aggressive ball returns and catching edge cases).
- Possible Causes:
- Limited training duration (~2.5M steps is good but still small for complete Pong mastery).
- Possibly suboptimal learning rate, rollout length, or exploration schedule.

Conclusion

At 2.5 million steps, your Actor-Critic agent on Pong-v5 has moved beyond random play, showing:
- Slightly competitive behavior,
- High policy stability (low variance across games),
- Signs of systematic learning (reduced entropy and consistent returns).

CONTRIBUTION SUMMARY

TEAM MEMBER	CONTRIBUTION
Aditi Sinha	50%
Suman Saurav	50%

REFERENCES

- Mnih et al., “Asynchronous Methods for Deep Reinforcement Learning” https://arxiv.org/abs/1602.01783
- Gymnasium Docs: https://gymnasium.farama.org/
- PyTorch Docs: https://pytorch.org/docs/stable/
- Gymnasium environments
- Atari Environments
- Lecture slides and Assignment 3 pdf
- Synchronous Methods for Deep Reinforcement Learning